

Evaluating Your Model 🤖

When building a model, especially one with many features (making it hard to visualize), you need a systematic way to measure its performance. Simply fitting the data well isn't enough, as a model can **overfit** (fit the training data perfectly but fail to generalize to new data).

The Train-Test Split

The standard procedure is to split your dataset into two parts:

1. **Training Set (e.g., 70%–80%)**: The majority of the data. This portion is used to **train** the model (i.e., find the optimal parameters w and b).
2. **Test Set (e.g., 20%–30%)**: The remaining data. This portion is **held back** and used *only* to **evaluate** the trained model's performance on unseen data.

Notation

- **Training Set:**
 - m_{train} : Number of training examples.
 - Examples: $(x^{(1)}, y^{(1)}), \dots, (x^{(m_{\text{train}})}, y^{(m_{\text{train}})})$
 - **Test Set:**
 - m_{test} : Number of test examples.
 - Examples: $(x_{\text{test}}^{(1)}, y_{\text{test}}^{(1)}), \dots, (x_{\text{test}}^{(m_{\text{test}})}, y_{\text{test}}^{(m_{\text{test}})})$
-

Evaluation for Regression (e.g., Linear Regression)

1. **Train the Model**: Find parameters w and b by minimizing the **cost function**, $J(w, b)$, on the **training set**.
 - $J(w, b) = \frac{1}{2m_{\text{train}}} \sum_{i=1}^{m_{\text{train}}} (f_{w,b}(x^{(i)}) - y^{(i)})^2 + \text{regularization term}$
 2. **Evaluate on Training Set (Training Error)**: Compute the average error on the **training set**, typically *without* the regularization term.
 - $J_{\text{train}}(w, b) = \frac{1}{2m_{\text{train}}} \sum_{i=1}^{m_{\text{train}}} (f_{w,b}(x^{(i)}) - y^{(i)})^2$
 3. **Evaluate on Test Set (Test Error)**: Compute the average error on the **test set**, also *without* the regularization term. This is the primary measure of how well your model generalizes.
 - $J_{\text{test}}(w, b) = \frac{1}{2m_{\text{test}}} \sum_{i=1}^{m_{\text{test}}} (f_{w,b}(x_{\text{test}}^{(i)}) - y_{\text{test}}^{(i)})^2$
- An **overfit** model (like the 4th-order polynomial) will have a very low J_{train} but a very high J_{test} .
-

Evaluation for Classification (e.g., Logistic Regression)

For classification, you have two common ways to evaluate the model:

Method 1: Using the Loss Function

This is the same as regression, but using the logistic loss (cross-entropy) instead of squared error.

1. **Train the Model:** Minimize the regularized **cost function** $J(w, b)$ on the training set.
2. **Evaluate (Test Error):** Compute the average **test error**, J_{test} , using the logistic loss on the test set.

$$\circ J_{test}(w, b) = -\frac{1}{m_{test}} \sum_{i=1}^{m_{test}} \left[y_{test}^{(i)} \log(f_{w,b}(x_{test}^{(i)})) + (1 - y_{test}^{(i)}) \log(1 - f_{w,b}(x_{test}^{(i)})) \right]$$

3. **Evaluate (Training Error):** Compute the average **training error**, J_{train} , using the logistic loss on the training set.

Method 2: Misclassification Error (More Common)

A more intuitive metric for classification is the **misclassification error**, which is the *fraction* of examples the model got wrong.

- **Test Error (J_{test}):** The fraction of test set examples where the model's prediction $\hat{y}$ does not equal the actual label y_{test} .
 - *Example:* If the model misclassifies 3 out of 30 test examples, $J_{test} = \frac{3}{30} = 0.1$ (or 10%).
- **Training Error (J_{train}):** The fraction of training set examples that the model misclassified.

Splitting your data into training and test sets is the first step in systematically evaluating your model and, as you'll see next, automatically selecting the best model configuration.